PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
2026

Taller de rendimiento

Autores
Juan Felipe Gonzalez
Maria Alejandra Rodriguez
Marlon Jhoan Garcia Restrepo

Sistemas operativos

Profesor
John Corredor Franco

Fecha de entrega
April 27, 2026

1. Resumen

El presente informe analiza el rendimiento de dos estrategias para la multiplicación de matrices: el método tradicional de filas por columnas y el método de filas por matriz transpuesta. Para cada estrategia se desarrollan implementaciones paralelas empleando *fork* y *POSIX threads*, con el fin de comparar su comportamiento en diferentes escenarios de ejecución.

La evaluación permite observar cómo la localidad temporal y espacial de los datos influye en el desempeño de los algoritmos, así como el efecto que tienen las características del procesador y el mecanismo de paralelización sobre los tiempos de ejecución obtenidos.

2. Introducción

El análisis de rendimiento en sistemas de cómputo es una herramienta clave para comprender la eficiencia de los algoritmos y el uso de los recursos disponibles, especialmente en entornos donde se manejan operaciones de alto costo computacional. La multiplicación de matrices constituye un caso representativo de este tipo de problemas, debido a su complejidad y a su amplia aplicación en diferentes áreas de la ingeniería y la computación. En este sentido, resulta pertinente estudiar cómo diferentes formas de implementación, particularmente aquellas que incorporan concurrencia, impactan el tiempo de ejecución y el desempeño general del sistema. El problema central radica en determinar en qué medida la ejecución paralela mediante procesos e hilos logra mejorar el rendimiento frente a la ejecución secuencial, considerando además la influencia de variables como el tamaño de las matrices, la cantidad de hilos y las características del hardware.

Para abordar este problema, se emplea una metodología experimental basada en la medición y comparación de tiempos de ejecución. Se diseñó una batería de experimentos en la que se definen distintos escenarios de prueba variando parámetros como la dimensión de las matrices y el número de hilos, ejecutando cada configuración múltiples veces con el fin de obtener valores promedio representativos, reduciendo así el efecto de la variabilidad del sistema haciendo uso de herramientas de automatización para la ejecución de pruebas y recolección de datos, facilitando su posterior análisis mediante tablas y representaciones gráficas.

3. Objetivos

3.1. Objetivo general

Comparar el rendimiento de diferentes implementaciones paralelas de multiplicación de matrices, considerando el método de filas por columnas y el método de filas por matriz transpuesta, ejecutados mediante *fork* y *POSIX threads* en distintos equipos de cómputo.

3.2. Objetivos específicos

- Preparar equipos de cómputo con distintas características de hardware para la ejecución de las pruebas.

- Configurar la comunicación mediante SSH con los equipos de la universidad.
- Ejecutar los algoritmos de multiplicación de matrices bajo diferentes modelos de paralelización.
- Organizar los resultados obtenidos en carpetas identificadas con el formato `Soluciones_x`.
- Comparar los tiempos de ejecución obtenidos para cada algoritmo, equipo y configuración.
- Analizar la influencia de la localidad temporal y espacial, las características del procesador y el modelo de paralelización en el rendimiento obtenido.

4. Algortimos

La multiplicacion de matrices es un proceso donde se cruzan sus valores. Para poder multiplicar dos matrices, el numero de columnas de la primera matriz debe ser igual al numero de filas de la segunda matriz.

Por ejemplo, si se tiene:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Entonces, para hallar cada posicion de la matriz resultado, se toma una fila de la primera matriz y se multiplica por una columna de la segunda matriz:

$$A \times B = \begin{pmatrix} (1)(5) + (2)(7) & (1)(6) + (2)(8) \\ (3)(5) + (4)(7) & (3)(6) + (4)(8) \end{pmatrix}$$

Resolviendo las operaciones:

$$A \times B = \begin{pmatrix} 5 + 14 & 6 + 16 \\ 15 + 28 & 18 + 32 \end{pmatrix}$$

$$A \times B = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

En general, si una matriz A tiene tamanio $m \times n$ y una matriz B tiene tamanio $n \times p$, entonces el resultado de $A \times B$ sera una matriz de tamanio $m \times p$.

$$A_{m \times n} \times B_{n \times p} = C_{m \times p}$$

Este proceso es importante porque permite combinar informacion entre filas y columnas. En programacion, la forma en que se recorren estas filas y columnas puede afectar el rendimiento, ya que acceder a los datos por filas suele ser mas eficiente que acceder a ellos por columnas debido a la organizacion de la memoria.

A algunos programadores ingeniosos se les ocurrio una forma de optimizar la multiplicacion

de matrices: en lugar de recorrer directamente las columnas de la segunda matriz, primero calculan su transpuesta.

La transpuesta de una matriz consiste en convertir sus filas en columnas y sus columnas en filas. Por ejemplo:

$$B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Su transpuesta es:

$$B^T = \begin{pmatrix} 5 & 7 \\ 6 & 8 \end{pmatrix}$$

La idea es que, en la multiplicacion tradicional, cada elemento del resultado se calcula usando una fila de A y una columna de B:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

Pero al usar la transpuesta de B, esa columna de B se convierte en una fila de B^T. Entonces el calculo se puede expresar como:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B^T_{jk}$$

Es decir, en vez de recorrer:

$$\text{fila de } A \times \text{columna de } B$$

se recorre:

$$\text{fila de } A \times \text{fila de } B^T$$

Esto no cambia el resultado matemático, pero si mejora el rendimiento del programa. La razon es que en lenguajes como C, las matrices normalmente se almacenan por filas en memoria. Por eso, recorrer una fila completa suele ser mas rapido que recorrer una columna. Más adelante se profundiza las diferencias principales entre ambas metodologias.

4.1. Paralelismo con `fork()`

La funcion `fork()` permite crear nuevos procesos a partir de un proceso principal. Cuando se ejecuta `fork()`, el sistema operativo genera un proceso hijo que continúa ejecutando el mismo programa desde el punto donde fue creado.

En este trabajo, `fork()` se utiliza para dividir la multiplicacion de matrices por rangos de filas. Cada proceso hijo recibe una parte de la matriz resultado AB y calcula únicamente las filas que le corresponden. De esta forma, el trabajo total se reparte entre varios procesos.

Una caracteristica importante de `fork()` es que cada proceso tiene su propio espacio de memoria. Esto significa que, aunque el proceso hijo recibe una copia de las matrices, los

cambios que realiza no se reflejan directamente en la memoria del proceso padre, a menos que se utilicen mecanismos adicionales como memoria compartida.

Desde el punto de vista del rendimiento, `fork()` puede aprovechar varios nucleos del procesador, ya que los procesos pueden ejecutarse en paralelo. Sin embargo, crear procesos suele ser mas costoso que crear hilos, porque el sistema operativo debe administrar espacios de memoria independientes y realizar cambios de contexto entre procesos.

Por esta razon, en los algoritmos evaluados se espera que `fork()` pueda mejorar el rendimiento al repartir el trabajo, pero tambien puede presentar sobrecarga cuando se crean demasiados procesos o cuando el tamaño de la matriz no justifica el costo de paralelizar.

4.2. mxmForkFxC

El algoritmo en**mxmForkFxC.c** implementa la multiplicacion clasica de matrices usando el metodo de **filas por columnas**. En este enfoque, cada elemento de la matriz resultado AB se obtiene tomando una fila de la matriz A y multiplicandola por una columna de la matriz B.

Matematicamente, cada posicion AB_{ij} se calcula de la siguiente forma:

$$AB_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj}$$

Esto significa que para calcular un elemento de AB, el algoritmo recorre todos los valores de una fila de A y todos los valores correspondientes de una columna de B, multiplicandolos entre si y acumulando el resultado.

En el codigo, las matrices se almacenan como arreglos lineales de tipo `double`. Por eso, para acceder a la posicion de una matriz se usa la expresion:

```
matriz[N*fila + columna]
```

Por ejemplo, el valor ubicado en la fila f y columna c de una matriz se obtiene con:

```
matriz[N*f + c]
```

El programa tambien utiliza paralelismo mediante `fork()`. Primero, divide las filas de la matriz entre varios procesos. La variable `filasxP` calcula cuantas filas debe procesar cada proceso:

```
filasxP = N / num_P
```

Luego, cada proceso hijo calcula un rango de filas de la matriz resultado C. Para esto se definen una fila inicial y una fila final:

```
filaI = i * filasxP
```

```
filaF = (i == num_P - 1) ? N : filaI + filasxP
```

De esta manera, el ultimo proceso toma las filas restantes en caso de que el tamaño de la matriz no sea divisible exactamente entre el numero de procesos.

Hipotesis sobre problemas de rendimiento: aunque el algoritmo reparte el trabajo entre varios procesos, puede presentar limitaciones importantes. La principal dificultad esta en la forma en que accede a la matriz B. Mientras la matriz A se recorre por filas, la matriz B se recorre por columnas. En lenguajes como C, las matrices se almacenan por filas en memoria, por lo que acceder a una columna implica saltar entre posiciones no contiguas de memoria.

Esto puede afectar el rendimiento porque se aprovecha menos la memoria cache del procesador. En otras palabras, el procesador puede cargar datos cercanos esperando que se usen pronto, pero al recorrer columnas de B, los accesos estan mas separados. Como consecuencia, pueden generarse mas fallos de cache y el tiempo de ejecucion puede aumentar.

Adicionalmente, el uso de `fork()` crea procesos independientes. Esto puede tener un costo mayor que usar hilos, ya que cada proceso tiene su propio espacio de memoria. Aunque los procesos hijos calculan partes de `matC`, los cambios no necesariamente se comparten directamente con el proceso padre, a menos que se use memoria compartida o algun mecanismo adicional. Por eso, este enfoque puede servir para medir tiempos o imprimir resultados desde cada hijo, pero puede no ser suficiente si se espera que el padre tenga la matriz `C` completa al final.

En conclusion, este algoritmo realiza correctamente la multiplicacion clasica de matrices por filas y columnas, pero su rendimiento puede verse afectado por dos factores principales: el acceso no contiguo a las columnas de la segunda matriz y el costo de crear procesos con `fork()`.

4.3. mxmForkFxT

Este algoritmo implementa una optimizacion de la multiplicacion clasica de matrices usando **la transpuesta de la segunda matriz**. A diferencia del metodo filas por columnas, aqui no se recorre directamente la columna de B, sino una fila de B^T.

Primero, el programa reserva memoria para cuatro matrices:

$$A, \quad B, \quad C, \quad B^T$$

En el codigo, la matriz transpuesta se almacena en `matT`:

```
double *matT = calloc(N*N, sizeof(double));
```

Luego, antes de crear los procesos hijos, se calcula la transpuesta de la matriz B:

```
matrixTRP(N, matB, matT);
```

Esto significa que las columnas de B pasan a ser filas en `matT`. Por ejemplo, si:

$$B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Entonces:

$$B^T = \begin{pmatrix} 5 & 7 \\ 6 & 8 \end{pmatrix}$$

La multiplicacion sigue produciendo el mismo resultado matematico que el algoritmo tradicional. Cada posicion de C se calcula como:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj}$$

Pero usando la matriz transpuesta, el acceso se transforma en:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B^T_{jk}$$

En terminos de programacion, esto permite multiplicar:

$$\text{fila de } A \times \text{fila de } B^T$$

en lugar de:

$$\text{fila de } A \times \text{columna de } B$$

El programa tambien usa `fork()` para dividir el trabajo entre varios procesos. La cantidad de filas asignadas a cada proceso se calcula con:

```
filasxP = N / num_P
```

Cada proceso hijo recibe un rango de filas:

```
filaI = i * filasxP
```

```
filaF = (i == num_P - 1) ? N : filaI + filasxP
```

Luego cada hijo ejecuta:

```
mxmForkFxT(matA, matT, matC, N, filaI, filaF);
```

Es decir, cada proceso calcula una parte de la matriz resultado C, usando la matriz original A y la transpuesta `matT`.

Hipotesis sobre su mejora de rendimiento: este algoritmo puede ser mas eficiente que el metodo filas por columnas porque mejora el acceso a memoria. En C, los arreglos que representan matrices suelen almacenarse por filas. Por eso, recorrer una fila completa implica acceder a posiciones contiguas de memoria.

En el metodo tradicional, el algoritmo recorre filas de A, pero columnas de B. Esto genera saltos en memoria al acceder a B. En cambio, al usar la transpuesta, las columnas de B se convierten en filas de B^T, permitiendo que tanto A como B^T se recorran por filas.

Esto puede mejorar el uso de la memoria cache del procesador, ya que los datos cercanos en memoria tienen mas probabilidad de ser reutilizados rapidamente. Como consecuencia, se pueden reducir los fallos de cache y disminuir el tiempo de ejecucion.

Sin embargo, esta metodologia tambien tiene un costo adicional: antes de multiplicar, se debe calcular la transpuesta de B. Por esta razon, en matrices pequeñas, la ganancia puede no ser tan evidente. En matrices grandes, este costo inicial puede compensarse con una multiplicacion mas eficiente.

Adicionalmente, al igual que en el algoritmo anterior, el uso de `fork()` implica crear procesos independientes. Esto puede generar sobrecarga, ya que crear procesos es mas costoso que crear hilos. Tambien debe considerarse que cada proceso tiene su propio espacio de memoria, por lo que si se desea que el proceso padre conserve la matriz resultado completa, se necesita memoria compartida u otro mecanismo de comunicacion.

En conclusion, este algoritmo mantiene el mismo resultado matematico que la multiplicacion clasica, pero reorganiza la segunda matriz para mejorar el acceso a memoria. Su principal ventaja es que convierte el recorrido de columnas en un recorrido por filas, lo cual puede favorecer el rendimiento en arquitecturas donde la memoria cache tiene un papel importante.

4.4. Paralelismo con hilos POSIX

Los hilos POSIX, tambien conocidos como `pthreads`, permiten ejecutar varias tareas dentro de un mismo proceso. A diferencia de `fork()`, los hilos comparten el mismo espacio de memoria, por lo que pueden acceder a las mismas matrices globales sin necesidad de copiarlas.En este trabajo, los hilos POSIX se utilizan para dividir la multiplicacion de matrices por rangos de filas. Cada hilo calcula una parte de la matriz resultado AB, mientras los demas hilos trabajan sobre otras filas. Como cada hilo escribe en posiciones diferentes de la matriz resultado, no existe una competencia directa por la misma celda de memoria.El uso de `pthread_create()` permite crear los hilos, mientras que `pthread_join()` permite esperar a que todos terminen antes de continuar con el programa. Esto garantiza que la medicion de tiempo incluya la ejecucion completa del algoritmo.Desde el punto de vista del rendimiento, los hilos suelen ser mas ligeros que los procesos creados con `fork()`, ya que comparten memoria y requieren menor sobrecarga de administracion. Por esta razon, se espera que las versiones POSIX puedan presentar mejores tiempos en algunos casos.Sin embargo, el rendimiento tambien depende de otros factores, como la cantidad de hilos utilizados, el numero de nucleos disponibles, la distribucion de las filas y el patron de acceso a memoria. Si se crean mas hilos de los que el procesador puede manejar eficientemente, pueden aumentar los cambios de contexto y disminuir el beneficio del paralelismo.

4.5. mxmPosixFxC

Este algoritmo implementa la multiplicacion clasica de matrices usando el metodo de **filas por columnas**, pero en lugar de crear procesos con `fork()`, utiliza hilos POSIX mediante la libreria `pthread`.

La multiplicacion se realiza con la misma logica matematica del algoritmo clasico:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj}$$

Es decir, cada posicion de la matriz resultado C se obtiene multiplicando una fila de la matriz A por una columna de la matriz B.

En este codigo, las matrices `matrixA`, `matrixB` y `matrixC` se declaran como variables globales. Esto permite que todos los hilos puedan acceder a las mismas matrices dentro del mismo espacio de memoria, a diferencia de lo que ocurre con procesos creados con `fork()`.

Cada hilo recibe una estructura llamada `parametros`, donde se almacena:

`idH` : identificador del hilo

`nH` : numero total de hilos

`N` : tamanio de la matriz

Con estos datos, cada hilo calcula el rango de filas que le corresponde procesar:

```
filaI = (D/nH) * idH
```

```
filaF = (D/nH) * (idH + 1)
```

De esta forma, el trabajo se divide por bloques de filas. Cada hilo calcula una parte de la matriz resultado C, recorriendo las columnas de B para cada fila asignada.

Dentro de la funcion `mxmPosixFxC`, el calculo se realiza con tres ciclos anidados. El primer ciclo recorre las filas asignadas al hilo, el segundo ciclo recorre las columnas de la matriz B, y el tercer ciclo realiza la suma de productos:

```
Suma += *pA * *pB
```

El puntero `pA` avanza de forma continua por la fila de A:

```
pA++
```

Mientras que el puntero `pB` avanza por la columna de B:

```
pB += D
```

Esto evidencia que el algoritmo sigue el metodo filas por columnas: la matriz A se recorre por filas, pero la matriz B se recorre por columnas.

Hipotesis sobre sus problemas de rendimiento: aunque el uso de hilos POSIX puede reducir la sobrecarga frente a los procesos `fork()`, este algoritmo todavia conserva el principal problema del metodo clasico: el acceso no contiguo a la segunda matriz.

En C, las matrices almacenadas como arreglos lineales se organizan por filas. Por eso, recorrer una fila implica acceder a posiciones consecutivas de memoria. Sin embargo, en este algoritmo la matriz B se recorre por columnas mediante:

```
pB += D
```

Esto significa que el programa salta de una fila a otra para acceder a los elementos de una misma columna. Como consecuencia, se aprovecha menos la memoria cache del procesador y pueden aumentar los fallos de cache.

Otro punto importante es que la division de filas no contempla completamente los casos en los que N no es divisible entre el numero de hilos. Como se calcula:

```
filaF = (D/nH) * (idH + 1)
```

si sobran filas, estas podrian no ser procesadas por ningun hilo. Esto podria afectar la correctitud del resultado cuando el tamanio de la matriz no se divide exactamente entre la cantidad de hilos.

Tambien se observa el uso de un `mutex`, pero no protege una seccion critica real del calculo. Cada hilo escribe en filas diferentes de `matrixC`, por lo que no hay conflicto directo de escritura. Por esta razon, el bloqueo y desbloqueo del `mutex` al final no aporta a la sincronizacion del calculo y puede considerarse innecesario.

En conclusion, este algoritmo paraleliza la multiplicacion clasica mediante hilos POSIX, lo cual puede ser mas ligero que usar procesos. Sin embargo, su rendimiento puede verse limitado por el acceso por columnas a la matriz B, por una posible mala distribucion de filas cuando N no es divisible entre el numero de hilos, y por el uso innecesario de un `mutex`.

4.6. mxmPosixFxT

Este algoritmo implementa la multiplicacion de matrices usando hilos POSIX, pero con una optimizacion importante: antes de multiplicar, calcula la **transpuesta de la segunda matriz**. Es decir, en lugar de trabajar directamente con B, crea una nueva matriz B^T, almacenada en el codigo como `matrixT`.

La multiplicacion matematica sigue siendo la misma:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj}$$

Sin embargo, al usar la transpuesta, la operacion se expresa como:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B^T_{jk}$$

Esto permite que el algoritmo multiplique:

$$\text{fila de } A \times \text{fila de } B^T$$

en lugar de:

$$\text{fila de } A \times \text{columna de } B$$

En el codigo, primero se reserva memoria para cuatro matrices:

$$A, \quad B, \quad C, \quad B^T$$

La matriz transpuesta se crea con la funcion:

```
matrixTRP(N, matrixB, matrixT);
```

Luego, se crean los hilos con `pthread_create`. Cada hilo recibe una estructura `parametros`, donde se almacena su identificador, el numero total de hilos y el tamanio de la matriz.

Cada hilo calcula un rango de filas de la matriz resultado:

```
filaI = (D/nH) * idH
```

```
filaF = (D/nH) * (idH + 1)
```

Dentro de la funcion `mxmPosixFxT`, el calculo se realiza con tres ciclos anidados. El primer ciclo recorre las filas asignadas de A, el segundo ciclo recorre las filas de B^T, y el tercer ciclo realiza la suma de productos.

En esta version, los punteros avanzan asi:

```
pA++
```

```
pB++
```

Esto significa que tanto la fila de A como la fila de B^T se recorren de forma continua en memoria.

Hipotesis sobre su mejora de rendimiento: este algoritmo deberia presentar un mejor rendimiento que la version filas por columnas con POSIX, porque mejora el patron de acceso a memoria. En C, las matrices almacenadas como arreglos lineales se organizan por filas. Por esta razon, recorrer una fila completa aprovecha mejor la memoria cache del procesador.

En la version filas por columnas, la matriz B se recorria con saltos mediante:

```
pB += D
```

En cambio, en esta version, la matriz transpuesta permite recorrer la segunda matriz de forma secuencial:

```
pB++
```

Esto reduce los accesos no contiguos a memoria y puede disminuir los fallos de cache. Como resultado, se espera que el tiempo de ejecucion sea menor, especialmente en matrices grandes.

Sin embargo, esta mejora tiene un costo adicional: antes de multiplicar, el programa debe calcular la transpuesta de B. En matrices pequeñas, ese costo puede hacer que la diferencia de rendimiento no sea tan evidente. En matrices grandes, la ganancia por un mejor uso de cache puede compensar el costo inicial de transponer la matriz.

Tambien se mantiene una posible limitacion en la distribucion de filas. El rango se calcula con division entera:

```
D/nH
```

Por eso, si D no es divisible exactamente entre el numero de hilos, algunas filas podrian quedar sin procesar. Para evitarlo, el ultimo hilo deberia tomar las filas restantes, como se hizo en la version con `fork()`.

Finalmente, el `mutex` aparece en el codigo, pero no protege una seccion critica real. Cada hilo escribe en filas diferentes de `matrixC`, por lo que no existe una competencia directa por la misma posicion de memoria. Por esta razon, el bloqueo y desbloqueo del `mutex` al final no aporta a la sincronizacion del calculo.

En conclusion, este algoritmo combina dos estrategias de mejora: el uso de hilos POSIX y el uso de la transpuesta de la segunda matriz. La principal ventaja es que permite recorrer ambas matrices por filas, aprovechando mejor la localidad espacial de la memoria y la cache del procesador.

5. Metodologia

En este trabajo usamos un enfoque experimental orientado a la medición del tiempo de ejecución de diferentes implementaciones del algoritmo de multiplicación de matrices. Para ello, se utilizó el script *lanzador.pl*, el cual fue modificado con el fin de definir los parámetros de experimentación, específicamente el tamaño de las matrices, el número de hilos y la cantidad de repeticiones por cada escenario.

5.1. Definición de niveles de concurrencia y esquema de repetición

En cuanto a la configuración de los experimentos, se establecieron valores de número de hilos correspondientes a 1,2,4,8,12, 32, con el objetivo de analizar el comportamiento del algoritmo bajo diferentes niveles de concurrencia. Adicionalmente, se fijó un total de 30 repeticiones por cada combinación de parámetros, con el propósito de obtener resultados promedio representativos y reducir la variabilidad en las mediciones.

5.2. Selección y escalamiento del tamaño del problema

Para el tamaño de las matrices, se seleccionaron dimensiones de 600,1200, 1800 y 2400, las cuales fueron evaluadas de manera independiente. Esta estrategia permitió optimizar el proceso de recolección de datos, ejecutando cada tamaño de matriz de forma secuencial y facilitando la organización de los resultados obtenidos.

5.3. Caracterización de las implementaciones

El estudio incluyó cuatro implementaciones del algoritmo de multiplicación de matrices: dos basadas en el uso de procesos (fork) y dos mediante hilos POSIX (pthreads). En cada caso, se evaluaron dos enfoques de cálculo: el método clásico de multiplicación fila por columna y

una variante optimizada que emplea la matriz transpuesta, con el fin de mejorar la localidad de memoria. De esta manera, se logró comparar tanto el modelo de concurrencia utilizado como la estrategia de acceso a datos.

5.4. Estrategia de adquisición, almacenamiento y análisis de datos

Los resultados de cada ejecución fueron almacenados en archivos con extensión .dat, generados automáticamente por el script. Estos archivos (.dat) contienen los tiempos de ejecución correspondientes a cada configuración evaluada, luego se realizo un codigo en el bash de linux que se encargo de ordenar las carpetas dependiendo el numero de hilos. A partir de dicha información devidamente organizada, se realizó un análisis estadístico comparativo, considerando variables como el sistema de cómputo utilizado, la cantidad de hilos, las características de hardware y los difrerentes algoritmos ulizados, con el fin de identificar patrones de comportamiento y diferencias en el rendimiento entre las distintas implementaciones.

6. Descripcion de los equipos

Para la evaluacion de rendimiento se utilizaron equipos con diferentes caracteristicas de procesamiento, memoria RAM, cantidad de hilos, nucleos y memoria cache. Estas diferencias pueden influir directamente en el comportamiento de los algoritmos de multiplicacion de matrices, especialmente porque estos dependen de la capacidad de procesamiento paralelo y del acceso eficiente a memoria.

6.1. Equipo 1: Ubuntu con Pop!_OS

Este equipo cuenta con arquitectura `x86_64`, 8 CPU logicas, 4 nucleos fisicos, 12 GB de RAM y 2 hilos por nucleo. Su frecuencia maxima es de 3.700 MHz y su frecuencia minima es de 400 MHz. Tiene 192 KiB de cache L1d, 128 KiB de L1i, 2 MiB de L2 y 6 MiB de L3.

Este equipo tiene una mejor capacidad de paralelismo que el anterior, ya que cuenta con 4 nucleos fisicos y 8 hilos logicos. Por esta razon, se espera que los algoritmos con POSIX y Fork puedan distribuir mejor el trabajo entre unidades de ejecucion.

La presencia de cache L3 tambien puede mejorar el rendimiento, ya que permite almacenar datos compartidos o reutilizados antes de tener que acceder a la RAM. En este equipo, la version con transpuesta podria mostrar una mejora importante, porque recorre tanto la matriz A como la matriz transpuesta B^T de forma mas continua en memoria.

6.2. Equipo 2: Ubuntu Jammy Jellyfish

Este equipo cuenta con arquitectura `x86_64`, 6 CPU logicas, 6 nucleos fisicos, 16 GB de RAM y 1 hilo por nucleo. No se reporta frecuencia maxima ni minima. Tiene 288 KiB de L1d, 192 KiB de L1i, 7.5 MiB de L2 y 42 MiB de L3.

Aunque tiene menos CPU logicas que el equipo anterior, posee 6 nucleos fisicos reales. Esto puede ser beneficioso porque cada hilo o proceso puede ejecutarse en un nucleo completo, sin depender de hyperthreading. Por esta razon, podria tener un rendimiento estable en ejecuciones paralelas con 6 hilos o procesos.

Su cache L3 de 42 MiB es considerablemente alta frente a los equipos anteriores. Esto puede favorecer mucho los algoritmos de multiplicacion de matrices, ya que una mayor parte de los datos puede permanecer cerca del procesador. Bajo la hipotesis de jerarquia de memoria, este equipo podria reducir accesos costosos a RAM y mejorar el rendimiento general.

6.3. Equipo 3: Ubuntu Noble con 20 CPU logicas

Este equipo cuenta con arquitectura `x86_64`, 20 CPU logicas, 12 nucleos fisicos, 46 GB de RAM y 2 hilos por nucleo. Su frecuencia maxima es de 4.900 MHz y su frecuencia minima es de 800 MHz. Tiene 512 KiB de L1d, 512 KiB de L1i, 12 MiB de L2 y 25 MiB de L3.

Este es uno de los equipos con mayor capacidad de procesamiento paralelo. Al tener 12 nucleos y 20 hilos logicos, se espera que pueda ejecutar con mejor rendimiento los algoritmos paralelos, especialmente cuando se usan cantidades de hilos cercanas al numero de nucleos o CPU logicas disponibles.

Su alta frecuencia maxima tambien puede favorecer el tiempo de ejecucion. Sin embargo, no necesariamente usar mas hilos siempre dara mejores resultados. Si se supera la cantidad eficiente de hilos, puede aparecer sobrecarga por sincronizacion, planificacion del sistema operativo o competencia por memoria cache.

En este equipo, la version filas por transpuesta podria beneficiarse bastante de la localidad espacial, ya que los accesos secuenciales a memoria permiten aprovechar mejor la cache. Tambien, al tener 46 GB de RAM, es menos probable que las matrices grandes generen problemas por falta de memoria principal.

6.4. Equipo 4: Ubuntu Noble con 16 CPU logicas

Este equipo cuenta con arquitectura `x86_64`, 16 CPU logicas, 8 nucleos fisicos, 16 GB de RAM y 2 hilos por nucleo. Su frecuencia maxima es de 4.465 MHz y su frecuencia minima es de 403 MHz. Tiene 256 KiB de L1d, 256 KiB de L1i, 4 MiB de L2 y 16 MiB de L3.

Este equipo tambien tiene una buena capacidad de paralelismo, aunque menor que el equipo de 20 CPU logicas. Sus 8 nucleos fisicos y 16 hilos pueden permitir una ejecucion eficiente de los algoritmos POSIX y Fork, especialmente si el numero de hilos se mantiene dentro de ese rango.

La cache L3 de 16 MiB puede ayudar a reducir accesos a RAM, pero al ser menor que la del equipo Jammy Jellyfish y la del equipo Noble de 20 CPU logicas, podria presentar mas fallos de cache en matrices grandes. Aun asi, por su frecuencia maxima alta, podria lograr buenos tiempos de ejecucion en matrices medianas.

6.5. Hipotesis general de rendimiento

De acuerdo con el principio de jerarquia de memoria, el procesador accede primero a las memorias mas cercanas y rapidas, como L1, L2 y L3. Si los datos no se encuentran alli, debe acceder a la RAM, que es mas lenta. Por eso, los algoritmos que aprovechan mejor la cache suelen tener mejor rendimiento.

En la multiplicacion filas por columnas, la matriz A se recorre por filas, pero la matriz B se recorre por columnas. Esto puede generar accesos no contiguos a memoria y afectar

la localidad espacial. En cambio, en la multiplicacion con transpuesta, tanto A como B^T se recorren por filas, lo que permite acceder a posiciones mas cercanas en memoria.

Por esta razon, se plantea la hipotesis de que los algoritmos con transpuesta tendran mejor rendimiento que los algoritmos filas por columnas, especialmente en matrices grandes. Esto se debe a que aprovechan mejor la localidad espacial y reducen los fallos de cache.

Tambien puede existir localidad temporal cuando un dato cargado en cache se reutiliza en un intervalo corto de tiempo. En la multiplicacion de matrices, esto puede ocurrir cuando ciertos valores de las matrices permanecen disponibles en cache mientras se calculan varios elementos de la matriz resultado.

Finalmente, se espera que los equipos con mas nucleos, mas hilos y mayor memoria cache tengan mejores resultados en las versiones paralelas. Sin embargo, el rendimiento no depende solamente del numero de hilos, sino tambien de la distribucion del trabajo, el costo de crear procesos o hilos, la cantidad de memoria cache disponible y la forma en que el algoritmo accede a los datos en memoria.

7. Diseño de pruebas

Para el desarrollo de las pruebas se instaló en cada uno de los equipos el directorio **evalRend2610**, el cual contiene los ejecutables, scripts y carpetas necesarias para ejecutar los algoritmos de multiplicación de matrices.

El archivo principal utilizado para automatizar las ejecuciones fue **lanzador.pl**. Este script fue configurado para ejecutar los cuatro algoritmos evaluados:

- `mxmPosixFxC`
- `mxmPosixFxT`
- `mxmForkFxC`
- `mxmForkFxT`

Cada algoritmo se ejecutó con diferentes cantidades de hilos o procesos:

$$1,\ 2,\ 4,\ 8,\ 12,\ 24,\ 32$$

Para cada combinación de algoritmo, tamaño de matriz y número de hilos/procesos, se realizaron **30 repeticiones**. Los resultados de cada ejecución se almacenaron en archivos independientes dentro de una carpeta específica llamada **Soluciones_x**, donde x representa el tamaño de la matriz evaluada.

Por ejemplo, para una matriz de tamaño 600, los resultados se almacenaron en la carpeta:

`Soluciones_600`

El nombre de cada archivo de salida conserva la siguiente estructura:

`nombreAlgoritmo-tamanio-Hilos-numeroHilos.dat`

Esto permitió organizar los resultados de forma separada y facilitar posteriormente el análisis de rendimiento.

Listing 1: Archivo lanzador.pl

```
#!/usr/bin/perl
#*************************************************************
#               Pontificia Universidad Javeriana
#     Autor: J. Corredor
#     Nombre Estudiante:
#     Fecha:
#     Materia: Sistemas Operativos
#     Tema: Taller de Evaluacion de Rendimiento
#     Fichero: script automatizacion ejecucion por lotes
#*************************************************************/

$Path = `pwd`;
chomp($Path);

@Nombre_Ejecutable = ("mxmPosixFxC","mxmPosixFxT","mxmForkFxC","mxmForkFxT");
@Size_Matriz  = (x); // tamanio de la matriz
@Num_Hilos    = (1,2,4,8,12,24,32); // numero de hilos o procesos
$Repeticiones = 30;
$Resultados   = "Soluciones_x"; // carpeta de resultados
$Binarios     = "bin";

foreach $nombre (@Nombre_Ejecutable){
    foreach $size (@Size_Matriz){
        foreach $hilo (@Num_Hilos) {
            $file = "$Path/$Resultados/$nombre-".$size."-Hilos-".$hilo.".dat";
            printf("$file \n");
            for ($i=0; $i<$Repeticiones; $i++) {
                system("$Path/$Binarios/$nombre $size $hilo  >> $file");
                printf("$Path/$Binarios/$nombre $size $hilo \n");
            }
            close($file);
        }
    }
}
```

Las pruebas se realizaron con matrices cuadradas de los siguientes tamaños:

$$600,\ 1200,\ 1800,\ 2400$$

Cada tamaño de matriz se ejecutó por separado. Esta decisión se tomó para mantener una mejor organización de los archivos de salida y para evitar una carga excesiva en los equipos durante la ejecución de las pruebas.

En los equipos de la universidad, las pruebas se ejecutaron de manera remota mediante conexión SSH. Debido a que algunos experimentos tomaban varias horas o incluso días, fue necesario utilizar el comando:

```
nohup ./lanzador.pl &
```

Este comando permite que el script continúe ejecutándose aunque se cierre la terminal o se pierda la conexión SSH. De esta forma, se evitó que la desconexión interrumpiera la recolección de resultados.

En resumen, el diseño de pruebas consistió en ejecutar los cuatro algoritmos sobre diferentes tamaños de matriz, variando la cantidad de hilos o procesos, realizando 30 repeticiones por caso y almacenando los resultados en archivos organizados por tamaño de matriz y configuración de ejecución.

8. Resultados

A partir de los datos obtenidos, se identifican diferencias claras en el comportamiento de los sistemas evaluados, tanto en términos de desempeño máximo como en su capacidad de escalabilidad. En general, los resultados muestran que cada equipo presenta un punto óptimo de ejecución distinto, determinado por sus características de hardware y la forma en que gestionan la concurrencia. El sistema Noble1A alcanza el mejor desempeño global, con un speedup máximo de 6.49 utilizando 8 hilos, lo que lo posiciona como el equipo más balanceado. Su comportamiento evidencia una buena capacidad de escalabilidad hasta dicho punto, aunque posteriormente presenta una disminución en la eficiencia al incrementar el número de hilos. Por su parte, Jellyfish muestra un comportamiento orientado a cargas más exigentes, alcanzando su mejor resultado con un speedup de 6.36 a 12 hilos. Este sistema destaca por su capacidad de paralelismo en escenarios de alta carga, aunque presenta limitaciones cuando se incrementa excesivamente el número de hilos, evidenciando sobrecarga en configuraciones más altas. En el caso de Noble2M, se observa un rendimiento más estable pero con menor capacidad de escalamiento, alcanzando un speedup máximo de 5.84 con 8 hilos. Este comportamiento sugiere un sistema más conservador, con menor variabilidad en sus resultados, pero también con un techo de rendimiento más bajo en comparación con los otros equipos. Finalmente, el sistema basado en Pop!OS presenta un comportamiento competitivo, con un rango óptimo estimado entre 8 y 12 hilos, destacándose por el uso de un scheduler moderno que favorece la gestión de procesos. Sin embargo, su desempeño final depende en gran medida del hardware sobre el cual se ejecuta.

9. Analisis

Desde los resultados obtenidos, se observa que el rendimiento del algoritmo mejora al aumentar el número de hilos, especialmente en matrices de mayor tamaño. Aunque la carga computacional permite el mejor aprovechamiento de los recursos, este comportamiento no es progresivo porque existe un punto donde el incremento de hilos no genera mejoras significativas, en ocasiones incluso puede perjudicar los tiempos de ejecución. El fenómeno está vinculado a las limitaciones del hardware y la gestión concurrente del sistema. Además, las soluciones que usan hilos (pthreads) suelen obtener resultados mucho mejores que aquellas que usan procesos (fork). Esto sugiere que hay una gestión más eficiente de los recursos compartidos. Por otro lado, el tamaño de la matriz tiene efectos directos sobre el rendimiento, favoreciendo el uso concurrente en problemas de dimensiones más amplias. De igual forma las diferencias entre diferentes métodos de cálculo indican que la forma de acceder a los datos

también determina los tiempos alcanzados, posibilitando identificar las configuraciones más efectivas entre las distintas alternativas analizadas.

Tal como se planteó en la hipótesis del presente trabajo, el uso de principios de localidad de memoria influye directamente en el rendimiento de las implementaciones evaluadas. En particular, se consideró que aquellas estrategias de cálculo que optimizan el acceso a memoria tendrían un mejor desempeño frente a aquellas con patrones de acceso menos eficientes. El principio de localidad de memoria describe cómo los algoritmos acceden a los datos en la jerarquía de memoria, siendo un factor determinante en el tiempo de ejecución, especialmente en operaciones intensivas como lo son la multiplicación de matrices. Este principio se divide principalmente en dos componentes: localidad espacial y localidad temporal.

9.1. Localidad espacial

La localidad espacial hace referencia al acceso a posiciones de memoria contiguas. En este experimento, se evidenció al comparar la multiplicación clásica fila por columna con la implementación que utiliza la matriz transpuesta. En el método tradicional, el acceso a la segunda matriz se realiza por columnas, lo que genera saltos en memoria y un bajo aprovechamiento de la caché. En contraste, al emplear la matriz transpuesta, los accesos se realizan por filas, lo que permite trabajar con datos almacenados de manera contigua en memoria. Esta diferencia favorece el uso eficiente de la caché y reduce los tiempos de acceso a memoria, impactando positivamente el rendimiento, especialmente en matrices de mayor tamaño.

9.2. Localidad temporal

Por otro lado, la localidad temporal se refiere a la reutilización de datos que han sido accedidos recientemente. En la multiplicación de matrices, ciertos elementos son utilizados repetidamente dentro de los ciclos de cálculo. En el contexto del experimento, las implementaciones que presentan patrones de acceso más ordenados (como la versión con matriz transpuesta) que facilitan que estos datos permanezcan en caché durante más tiempo, reduciendo la necesidad de acceder nuevamente a la memoria principal.

Durante la experimentación, los resultados registrados en los archivos .dat, evidencian el impacto de estos principios en el desempeño de las implementaciones. A medida que aumenta el tamaño de las matrices y el nivel de concurrencia, la eficiencia en el acceso a memoria se vuelve un factor crítico, pudiendo potenciar o limitar los beneficios del paralelismo.

9.3. Speed up

El speedup es uno de los indicadores más relevantes en el análisis de rendimiento de algoritmos paralelos, ya que permite cuantificar la mejora obtenida al ejecutar un programa utilizando múltiples hilos o procesos en comparación con su versión secuencial.

Matemáticamente, el speedup se define como:

$$S = \frac{T_{sec}}{T_{par}} \tag{1}$$

donde T_{sec} corresponde al tiempo de ejecución con un solo hilo y T_{par} al tiempo obtenido al utilizar múltiples hilos. Para ello, se emplearon los tiempos promedio registrados en los archivos `.dat`, producto de las 30 repeticiones realizadas para cada configuración.

A partir de este cálculo, logramos cuantificar la mejora en el rendimiento al incrementar el número de hilos, evidenciando si la reducción en el tiempo de ejecución es proporcional al nivel de concurrencia. De esta manera, es posible analizar el comportamiento de escalabilidad del algoritmo, identificando si se aproxima a un speed up ideal (crecimiento lineal) o si, por el contrario, se presentan limitaciones asociadas a factores como el overhead, la contención de recursos o las características del hardware.

9.4. Overhead en la ejecución paralela

El overhead es el costo extra de usar paralelismo: crear y manejar hilos o procesos, sincronizar y compartir recursos. Se nota cuando subir de 2, 4, 8, 12 a 32 hilos no baja el tiempo en la misma proporción, porque parte del trabajo se va en administrar la concurrencia y no en calcular. Con matrices pequeñas pega más fuerte; en grandes se diluye. Entre implementaciones, fork suele costar más que pthreads por el manejo de procesos. En los .dat aparece claro cuando, al aumentar hilos, el tiempo deja de caer o incluso sube.

10. Conclusiones y recomendaciones

El análisis de los resultados evidencia que el paralelismo permite reducir los tiempos de ejecución al distribuir la carga de trabajo entre múltiples hilos, logrando un mejor aprovechamiento de los recursos del CPU, especialmente en matrices de mayor tamaño. En este tipo de casos, el volumen de operaciones es suficientemente alto para que el costo de administración de los hilos sea mínimo frente al trabajo computacional, lo que se traduce en mejoras más notables en el desempeño.

Sin embargo, este comportamiento presenta un límite. Al incrementar el número de hilos más allá de la capacidad del hardware, particularmente el número de núcleos disponibles, la mejora en el rendimiento se estabiliza o incluso disminuye. Esto se debe a la competencia por el uso del CPU y a los costos adicionales asociados a la gestión de múltiples hilos o procesos, lo que en algunos casos genera tiempos de ejecución mayores.

Por otra parte, en matrices de menor tamaño, el beneficio del paralelismo es reducido, ya que el costo de administración de la concurrencia representa una proporción significativa del tiempo total. En conjunto, los resultados demuestran que el rendimiento depende de un balance adecuado entre el tamaño del problema, la cantidad de hilos utilizados y la capacidad del sistema, evidenciando que un mayor nivel de paralelismo no siempre implica una mejora en el desempeño. El mejor rendimiento entre los equipos evaluados se alcanza en torno a los 8 hilos, evidenciando que este valor se aproxima al punto óptimo de ejecución para las configuraciones analizadas. En particular, Jellyfish presenta su mayor speedup promedio cercano a 6.36 con 12 hilos, mientras que Noble1A y Noble2M alcanzan sus mejores resultados con aproximadamente 6.49 y 5.84 utilizando 8 hilos, respectivamente. Esto indica que, aunque existe una ligera variación entre equipos, el beneficio del paralelismo tiende a estabilizarse en rangos similares, determinados por las características del hardware. En este sentido, aumentar

el número de hilos más allá de estos valores no garantiza mejoras adicionales y puede incluso afectar el desempeño, confirmando que existe un punto de saturación donde los recursos del sistema se utilizan al máximo. Por tanto, la selección adecuada del número de hilos resulta clave para lograr un rendimiento óptimo en la ejecución del algoritmo.

11. Referencias

Referencias

[1] Hennessy, J. L., & Patterson, D. A. (2011). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.

[2] Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann.

[3] Stallings, W. (2018). *Computer Organization and Architecture: Designing for Performance*. Pearson.

[4] Tanenbaum, A. S. (2009). *Modern Operating Systems*. Pearson.

[5] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*. Wiley.

[6] Bryant, R. E., & O'Hallaron, D. R. (2015). *Computer Systems: A Programmer's Perspective*. Pearson.

[7] Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*. Johns Hopkins University Press.

[8] Strang, G. (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge Press.

[9] The Open Group. (2018). *The Open Group Base Specifications Issue 7, 2018 Edition*. Disponible en: `https://pubs.opengroup.org/onlinepubs/9699919799.2018edition/`

[10] Linux Manual Pages. (s.f.). *fork(2) — Linux manual page*. Disponible en: `https://man7.org/linux/man-pages/man2/fork.2.html`

[11] The Open Group. (2018). *pthread_create — The Open Group Base Specifications*. Disponible en: `https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_create.html`

[12] The Open Group. (2018). *pthread_join — The Open Group Base Specifications*. Disponible en: `https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_join.html`